

COMET: Communication and Memory Co-Design for Fine-Grained AI Inference in MCM Accelerators

Taishu Sheng, Guangyu Sun[†], Dezun Dong

HiNA Lab, College of Computer Science and Technology, National University of Defense Technology

[†]School of Integrated Circuits, Peking University, and Beijing Advanced Innovation Center for Integrated Circuits
{sts,dong}@nudt.edu.cn, gsun@pku.edu.cn

Abstract—Chiplet-based architectures emerge as a promising approach to overcoming the physical and manufacturing constraints faced by monolithic chips, enabling the scalable integration of computing resources to meet the growing demands of AI workloads. However, efficient inter-chiplet communication still faces significant bottlenecks, especially under the fine-grained and bursty Direct Memory Access (DMA) request patterns generated by processing elements in modern AI tasks. Existing communication models and simulators fail to capture these characteristics, which limits the accuracy of performance analysis and the effectiveness of optimization strategies. These limitations hinder DMA-communication inefficiencies in chiplet-based AI systems and pose challenges for designing HPC architectures.

To address these challenges, we present the first comprehensive chiplet communication model that explicitly incorporates fine-grained DMA traffic observed in realistic AI workloads. Building on this model, we propose COMET, a novel framework that intelligently searches for optimal DMA request aggregation and memory address mapping strategies tailored to chiplet environments. COMET dynamically consolidates small DMA transfers to improve bandwidth utilization and reduce communication latency, while also adapting on-chip memory mapping to align with workload-specific dataflows. This mitigates synchronization overhead across diverse AI tasks. Compared with inference on conventional chiplet communication schemes, COMET achieves $1.1\times$ – $2.6\times$ speedup and $1.5\times$ – $4.4\times$ higher bandwidth utilization across different DNN and LLM workloads.

I. INTRODUCTION

The rapid advancement of large language models (LLMs) [1], [2], [3], and other AI workloads [4], [5], [6], [7] dramatically increases demands on computational power and memory bandwidth [8]. As traditional monolithic system-on-chip (SoC) designs [9], [10] face physical limitations such as Moore’s Law [11] slowdown and reticle size constraints, chiplet-based architectures emerge as a promising approach. By integrating multiple smaller dies through advanced packaging [12], [13], chiplet designs offer scalable compute capacity and design flexibility. Industry efforts, including Simba [14], DOJO [15], and Cerebras [16], demonstrate the potential of chiplet architectures to accelerate AI computations, while academic research [17], [18] explores various chiplet-inspired systems. Despite these advances, efficient inter-chiplet communication still faces critical challenges that limit the overall performance of AI systems.

In modern multi-chip-module (MCM) architectures, a variety of communication modes exist in chiplet-based architectures, as shown in Figure 1, such as core-to-core, PE-to-PE,

and PE-to-memory interactions, depending on the granularity and location of data exchange. To improve I/O efficiency and reduce context-switching overhead, earlier systems often rely on single-access read/write operations aligned with cache-line boundaries, using Memory-Mapped I/O (MMIO). Since cache lines are typically 64 bytes, in AI-centric workloads characterized by frequent data exchange, MMIO becomes inefficient due to its small data granularity and high access latency [19]. To address these limitations, most mainstream AI chiplet systems adopt DMA for inter-chiplet communication, as it supports bulk data movement without involving the CPU. While DMA offers significant performance advantages over MMIO for large-scale data movement, its efficiency diminishes when handling transfers smaller than 1MB, as commonly seen in small-batch inference workloads. In such cases, the bulk-oriented design becomes inefficient, leading to poor bandwidth utilization and added overhead unrelated to actual data transfer. In practice, each processing element (PE) independently generates DMA requests after computation, often resulting in a high volume of small, fragmented transfers [20]. These small DMA transactions suffer from substantial fixed protocol overheads, leading to suboptimal bandwidth utilization and increased communication latency [21]. Moreover, limited on-chip memory forces workload distribution across chiplets, amplifying inter-chiplet traffic and further aggravating the inherent mismatch between fine-grained data from PEs and coarse-grained DMA communication, thereby worsening overall efficiency.

Multiple diverse AI workloads, such as object detection [22], [23], and chatbots [24], may co-execute concurrently on the MCM architectures, as shown in Figure 1. These workloads can typically be classified into two broad categories, each with distinct characteristics. Convolutional Neural Network (CNN) tasks exhibit strong data dependencies and rigid execution flows, often constrained by synchronization barriers that force many PEs to wait for data write-back before forming contiguous memory blocks, resulting in idle time and reduced efficiency. Transformer-based workloads, especially those built on the multi-head attention (MHA) mechanism, enable more flexible computation: each head can be independently pipelined, and the weak inter-head dependencies provide scheduling freedom and higher PE utilization. Some layer-mapping strategies [18] attempt to align Deep Neural Network (DNN) layers with chiplet-level DRAM constraints, but fixed PE address

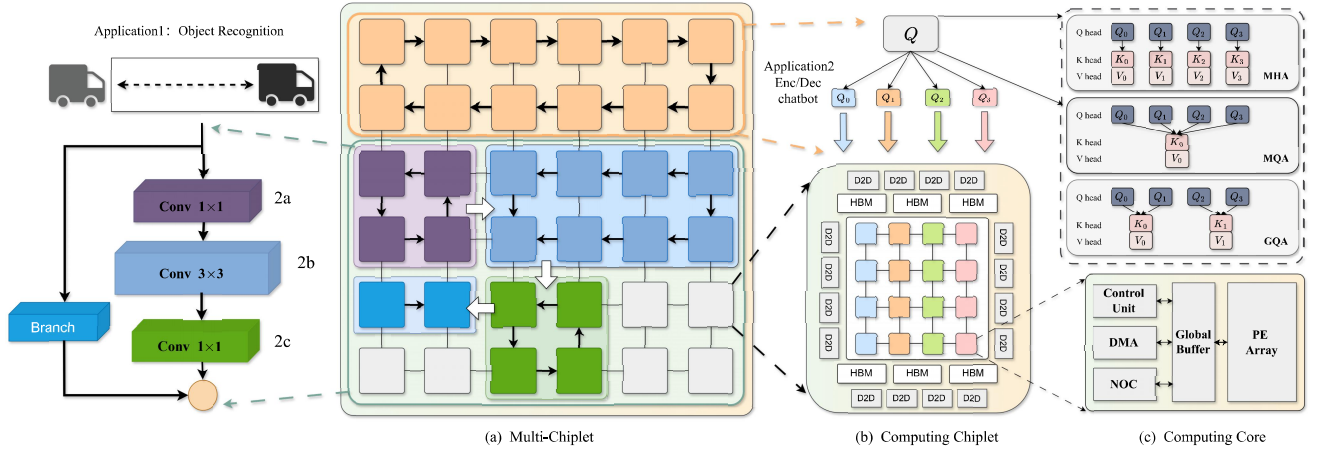


Fig. 1. This Figure explains the MCM architecture in three hierarchical levels: Multi-Chiplet, Computing Chiplet, and Core. It introduces two application scenarios: Application 1 based on CNN for Object Recognition, and Application2 based on the Transformer for dialogue and Encoding/Decoding applications.

mappings in conventional Conv accelerators [25] cannot exploit the flexibility of transformer-based models. Recent works [26], [27], [28] propose dynamic PE groupings to leverage this flexibility. However, practical issues [29], such as thermal effects or process-induced performance variability in PE, often offset the theoretical gains. Consequently, static on-chip memory address mapping schemes are increasingly ill-suited for heterogeneous and dynamic workloads, making it more difficult to determine when, how much, and which data to aggregate for DMA transmissions, ultimately increasing the complexity of communication scheduling and resource management. These challenges point to the need for adaptive dataflows and flexible mapping strategies tailored to multi-model co-execution on MCM platforms to ensure efficient end-to-end performance.

To address these challenges, we propose COMET tailored for chiplet-based AI architectures, focusing on efficient DMA request aggregation and adaptive workload-aware on-chip address mapping. Our framework improves bandwidth utilization by dynamically aggregating fine-grained DMA requests into larger transfers based on workload characteristics. Simultaneously, we design an adaptive address mapping strategy to accommodate heterogeneous AI workloads, reducing synchronization delays and improving dataflow efficiency.

Our main contributions are as follows:

- We present the first detailed chiplet communication model incorporating fine-grained DMA traffic under realistic AI workloads, providing a comprehensive analysis of the unique communication challenges in such scenarios.
- We propose a DMA request aggregation mechanism tailored for chiplet environments to enhance bandwidth utilization and reduce communication latency.
- We develop an adaptive on-chip memory address mapping strategy to better accommodate diverse AI workloads and alleviate synchronization overhead.
- We enhance the simulation framework by incorporating support for the DMA mechanism, facilitating more realistic chiplet communication modeling.

istic chiplet communication modeling.

Experiments on a variety of AI workloads, including DNNs and LLMs, show that COMET delivers a $1.1\times$ – $2.6\times$ end-to-end speedup and a $1.5\times$ – $4.4\times$ higher bandwidth utilization compared to inference using conventional chiplet communication schemes. When compared across different chiplet-based accelerators, COMET also achieves up to a $2.7\times$ end-to-end speedup.

II. BACKGROUND

A. Chiplet Technology Fundamentals

Chiplet technology, leveraging advanced packaging, offers a promising solution to the physical limits of monolithic chips by decomposing systems into smaller chiplets linked via high-bandwidth D2D interfaces (e.g., UCIE [30]), enabling process heterogeneity, better yield, and modular development. Widely adopted in AI accelerators, where large computational arrays tackle massive matrix operations as model complexity increases, this architecture distributes workloads across chiplets, with intermediate data exchanged via DMA-driven transfers over chiplet interconnects.

The architecture of the universal and configurable hardware template is shown in Figure 1(a), which is derived from extracting common features from existing chiplet accelerators [14], [15] and large-scale accelerators [31], [32]. The hardware template consists of three layers of computing nodes: Computing Chiplets, Computing Cores, and PEs, where each PE contains MAC arrays for computation and a vector unit for performing activations. A mesh NoC connects all Computing Cores across the Chiplets, with PEs inside each Core linked in a mesh as well, allowing for arbitrary PE-to-PE, Core-to-Core, Core-to-DRAM, and DRAM-to-Core communication. For inter-chiplet communication, the D2D transmission (TX) within a chiplet independently encodes the data and forwards it to the corresponding D2D reception (RX) in another chiplet. The D2D RX decodes the data and proceeds with NoC transmission. This inter-chiplet communication is fully automatic

and transparent to both the source and the destination. This hardware template and the corresponding hardware model can be easily modified to support various topologies [18]. Considering that most existing tiled accelerators [14], [15], [31], [33] currently adopt mesh interconnect networks, we also default to using the mesh topology in this template.

B. AI workloads

AI workloads span tasks such as image classification [34], object detection [22], and chatbots [24], typically using CNNs for vision tasks and Transformer models for natural language processing and sequence modeling. Each type has different computational patterns and hardware requirements.

CNN-based models rely on spatial convolutions, where each output activation is computed from local areas of the input feature map. A typical 2D convolution for output channel k at location (i, j) is:

$$Y_k(i, j) = \sum_{c=1}^{C_{in}} \sum_{m=1}^{K_H} \sum_{n=1}^{K_W} W_k(c, m, n) \cdot X_c(i + m, j + n)$$

Here, X is the input tensor, W is the kernel, and C_{in} is the number of input channels. Although convolutions can be converted to GEMM using im2col [35], it incurs high memory and bandwidth costs. Therefore, modern accelerators [14] generally prefer direct convolution implementations.

When Conv layers span multiple chiplets, synchronization delays and bursty communication arise as each chiplet processes part of the output. This increases the need for low-latency, high-bandwidth interconnects, especially with residual connections, which further increase communication overhead and pressure on inter-chiplet interconnects, as shown in Figure 1.

Transformer-based models that are built around the self-attention mechanism differ in their computations. For each input token, the query (Q), key (K), and value (V) matrices are computed:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where X is the input token embeddings and W_Q, W_K, W_V are projection matrices. The self-attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Transformers employ multi-head attention (MHA), where multiple heads run in parallel. Variants such as MQA and GQA offer flexible computation and memory access patterns. Since attention heads are independent, computations can be partitioned across chiplets. Each head can independently proceed as soon as it finishes, enabling asynchronous and continuous inter-chiplet dataflow, as shown in Figure 1.

C. DMA-based communication in AI accelerators

Mainstream AI chiplets [36], [37], [14], [15] predominantly employ DMA engines to perform large, autonomous transfers between memory and communication endpoints. DMA enables

asynchronous communication by offloading transfer logic from the host processor, allowing data movement to overlap with ongoing computations. This reduces control overhead, improves throughput, and is especially beneficial in chiplet-based designs where inter-tile communication must scale efficiently.

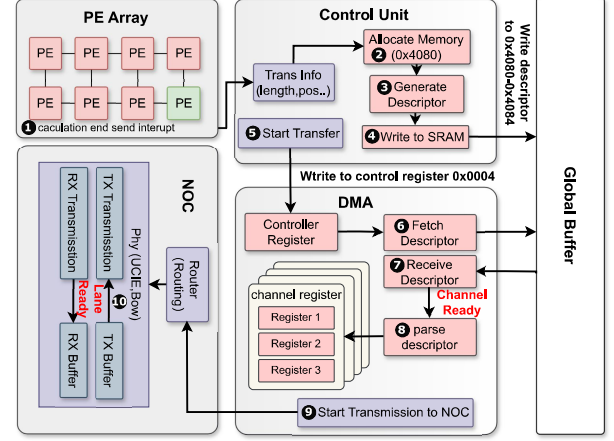


Fig. 2. This Figure illustrates how the PE triggers DMA communication within the on-chip network after completing an assigned task.

Figure 2 illustrates the DMA-based communication flow from the PE array to the on-chip network. The process begins when a PE finishes its computation and raises an interrupt to the control unit (1). Upon receiving the interrupt, the control unit first allocates a memory region for the outgoing data (step 2), and then generates a DMA descriptor encoding parameters such as transfer length, memory addresses, and destination coordinates (3). This descriptor is subsequently written into a dedicated region in the Global Buffer (4). Once the descriptor is written, the control unit initiates the data transfer by signaling the DMA engine (5), which involves writing to a control register (e.g., address 0x0004). The DMA engine then fetches the descriptor from the Global Buffer (6) and stores it locally (7). After parsing the descriptor (8) and verifying its correctness, the engine selects the idle channel and writes the parser result to the channel registers (9). The Channel then proceeds to initiate data transmission. The data were Routed through the on-chip router for packaging and then sent through the PHY (e.g., UCIe, BoW) to the destination chiplet (10).

III. RELATED WORK

A. DMA-Based Modification Mechanism

The existence of fixed communication overheads in DMA has been validated in prior studies [38], revealing bottlenecks that limit overall system performance. Several efforts have attempted to mitigate this, such as accelerating memory copy (e.g., Fastmove [39]), implementing hardware-based data re-ordering (e.g., DMA++ [40]), or offloading matrix operations [41] to DMA controllers for AI acceleration. However, these

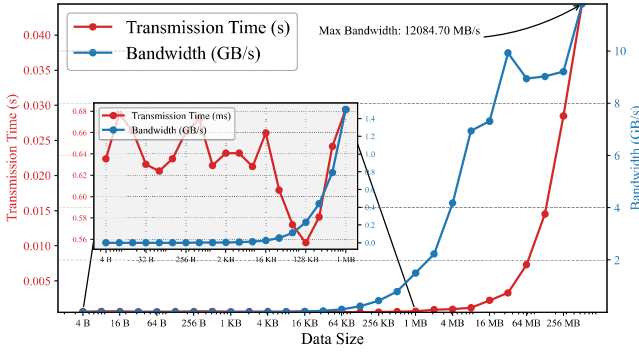


Fig. 3. Measured DMA bandwidth and transfer latency versus data size on the AMD Alveo™ U200 FPGA accelerator. A significant performance gap is observed between the theoretical peak and the actual achieved bandwidth, particularly for small data transfers.

approaches often rely on customized hardware extensions, restricting their applicability to specific scenarios. In contrast, our work targets general-purpose use cases by leveraging these inherent DMA bottlenecks without introducing hardware modifications.

B. Network-on-Chip Simulation Framework

Early on-chip network simulations relied on Gem5 [42] and BookSim [43]. Gem5, a full-system, cycle-accurate simulator integrating models like Garnet [44] and HeteroGarnet [45], supports up to 64 nodes but suffers from long runtimes and limited adaptability to AI-specific workloads. BookSim is lightweight and validated at RTL, scaling up to 10,000 nodes, with extensible interfaces. Recent simulators like CNSim [46] reach 279,040 nodes but simplify communication modeling. For AI workloads, ScaleSim [47] models GEMM, convolution, and FC layers using analytically derived compute cycles validated by RTL, and includes SRAM configuration support. Given the tight coupling of compute and communication in AI, this work adopts a recently developed framework [48] that integrates BookSim and ScaleSim, both RTL-validated, and extends it with DMA support and functional inference capabilities.

IV. MOTIVATION

A. Analysis of DMA-based AI workload communication

We begin by analyzing the characteristic of DMA-based communication. As shown in Figure 3, we evaluate DMA performance with different data sizes on an AMD Alveo™ U200 accelerator. The results show a significant gap between theoretical peak throughput and effective bandwidth, particularly for small data sizes. For data below 1 MB, bandwidth drops sharply, with transfer latencies dominated by fixed overheads, such as initialization and setup costs.

The setup overhead, highlighted in red (i.e., ②④⑤⑥⑦⑧⑨⑩ in Figure 2), consists of segments sensitive to queue lengths and the number of DMA controllers. For instance, segments ⑧⑩ are affected by queue lengths, while segment ⑥ correlates with the number of DMA controllers. According to the AXI-DMA documentation [49], a typical AXI-DMA instance

supports up to 16 channels, but only one controller handles control commands. The number of DMA controllers is limited by design trade-offs, such as silicon area, power, and cost, restricting DMA parallelism. This inefficiency is a key limitation for handling fine-grained memory transactions in conventional DMA mechanisms.

For AI workloads, CNNs and Transformer models differ significantly in inter-layer data dependencies and parallelism. CNNs have strong synchronization across spatial domains, leading to bursty communication, while Transformers allow for independent attention head computation, resulting in more sustained, asynchronous data movement. These differences emphasize the need for chiplet interconnects that support low-latency, high-throughput transmission for small and irregularly sized data, which traditional network-on-chip designs are not optimized for.

B. Chances and Challenges of AI accelerators communication

C1: DMA-based communication should match the fine-grained data transfer demands of AI Chips. Most existing DMA engines are primarily designed for large-scale data transfers and are seldom optimized for fine-grained communication scenarios, particularly in chiplet-based architectures. In traditional DMA-dominated applications, small data packets, often representing notifications or fragmented data, constitute only a minor portion of the overall traffic and are typically ignored in DMA optimization. However, in AI chiplet contexts, such fine-grained transfers occur more frequently and exert a critical impact on performance. Currently, the dominant mechanisms for on-chip communication are MMIO and DMA: MMIO offers fine granularity but limited throughput, while DMA requires large transfer sizes to amortize setup and queuing overhead.

As specified in the AXI-DMA documentation [49], a single descriptor can reference up to 256 MB of data using 28-bit address encoding. When these descriptors are used repeatedly to transfer only a few KB, the associated overhead significantly degrades throughput. This mismatch exposes a critical gap between AI workloads' fine-grained data demands and the coarse-grained behavior of conventional DMA. Therefore, there is a promising opportunity to develop fine-grained, workload-aware DMA methods for AI chips that bridge the gap between MMIO and traditional DMA, improving bandwidth utilization and boosting on-chip communication efficiency in chiplet-based AI systems.

C2: Static address mapping should adapt to the dynamic workload demands of AI chips. Current chiplet systems frequently employ fixed address mapping schemes, where the address space is first assigned sequentially from chiplets to cores, and then from cores to PEs in a sequential manner. This rigid mapping strategy fails to adapt to the highly dynamic and heterogeneous memory access patterns of AI workloads. For instance, Conv layers exhibit strong spatial reuse, but they also generate intermediate tensors with complex shapes that must be shared across chiplets. Though some frameworks convert convolutions to GEMMs using transformations like im2col,

such conversions significantly increase memory overhead, which is undesirable for on-chip accelerators. Furthermore, in transformer workloads, the token-based parallelism results in asymmetric memory usage among PEs. When multiple workloads co-exist, or when execution stalls due to unbalanced PE behavior, fixed mappings can cause delays in data aggregation and increase communication pressure. A mapping strategy that adapts to workload characteristics is critical for efficient resource utilization and data movement.

V. COMET FRAMEWORK

A. Problem Formulation

To the best of our knowledge, this is the first work to explicitly incorporate DMA-based communication, based on prior studies [18], [14], and the AXI-DMA specification, into the modeling of inter-chiplet scheduling for AI workloads. Unlike previous abstractions [46] that assume ideal or uniform data movement, our approach accounts for realistic hardware constraints, such as limited DMA controller availability and task-level contention, which impact latency and throughput.

We consider a heterogeneous chiplet system with multiple computing cores $c \in \mathcal{C}$, each having a limited number of PEs and DMA controllers. These resources are shared across concurrent tasks, leading to contention that must be modeled during scheduling. The workload is represented as a set of tasks $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$, where each task t_i involves a DMA transfer between cores and associated local computation. A task t_i is modeled as a tuple $(c_i^{\text{src}}, p_i^{\text{src}}, c_i^{\text{dst}}, p_i^{\text{dst}}, r_i)$, where c_i^{src} and c_i^{dst} are the source and destination cores, p_i^{src} and p_i^{dst} are the processing elements, and r_i is the request's issue timestamp.

a) Computation Modeling: The compute time of each task may vary due to hardware-level noise [29] such as process variation or dynamic voltage and thermal conditions. These effects are particularly relevant in advanced chiplet-based systems, where manufacturing variation and physical layout differences can lead to per-core performance discrepancies. To model this, we introduce a PE-level perturbation factor $\delta_{c,p} > 0$ that scales the ideal execution time $\hat{\tau}_i$ of a task on a particular PE. The resulting effective computation time is given by:

$$\tau_i^{\text{comp}} = \hat{\tau}_i \cdot \delta_{c_i^{\text{src}}, p_i^{\text{src}}} \quad (1)$$

b) DMA-Based Communication Modeling: Communication occurs via DMA engines, which perform inter-chiplet transfers. Each transfer proceeds in several stages:

- 1) **DMA Request Notification:** After local computation, the source PE triggers an interrupt to notify the controller of the pending DMA transfer, including source/destination addresses and data size. The overhead of interrupt handling and memory allocation is modeled as a constant time $\bar{\tau}_{\text{generate}}$.
- 2) **Control Path Routing and Descriptor Construction:** The notification is routed to the controller via the on-chip mesh network with routing delay $H_i \cdot \lambda_{\text{hop}}$, where H_i is the Manhattan distance. The controller constructs the

DMA descriptor and notifies the DMA controller to fetch it.

- 3) **DMA Controller Queuing:** After receiving the descriptor, the DMA controller may queue it behind existing requests. The queuing delay is approximated as $\tau_i^{\text{queue}} = q_i \cdot \bar{\tau}_{\text{launch}}$, where q_i is the number of pending descriptors and $\bar{\tau}_{\text{launch}}$ is the average descriptor handling latency.
- 4) **Launch Time:** The DMA launch time includes memory allocation for the descriptor and data, interrupt handling, and the DMA controller reading and programming the descriptor. This incurs a constant configuration cost $\gamma = \bar{\tau}_{\text{generate}} + \bar{\tau}_{\text{launch}}$.

Combining these stages, the complete launch overhead is:

$$\tau_i^{\text{launch}} = H_i \cdot \lambda_{\text{hop}} + q_i \cdot \bar{\tau}_{\text{launch}} + \gamma \quad (2)$$

c) Transmission Modeling: After launch, the data is transmitted via a UCle-based inter-chiplet interconnect. We adopt the standard α - β model:

$$\tau_i^{\text{transmit}} = \alpha + \beta \cdot N_i \quad (3)$$

where N_i is the payload size. We assume a conservative UCle latency value of $\alpha = 8$ ns, consistent with the measured single-hop latency in Short-Reach UCle PHY implementations [50], and β is the inverse of aggregate chiplet bandwidth.

The total DMA communication latency is:

$$\tau_i^{\text{comm}} = \tau_i^{\text{launch}} + \tau_i^{\text{transmit}} \quad (4)$$

d) Collective Aggregation Modeling: In many DNN operators, such as convolutions with spatial tiling or residual fusion, intermediate activations must be aggregated across chiplets before being consumed by the next layer. Let $\mathcal{S} \subseteq \mathcal{C}$ denote the set of chiplets participating in the aggregation, and let $P = |\mathcal{S}|$.

We model the aggregation delay as a function of both the algorithm A employed and the participating chiplet group $\mathcal{S}$:

$$\tau_i^{\text{agg}} = \tau^{\text{agg}}(A, \mathcal{S}) \quad (5)$$

To integrate aggregation behavior into the task execution semantics, we model each collective as an auxiliary synchronization node a_g in the task graph. This node becomes executable only after all contributing tasks $\{t_k\}_{k \in \mathcal{S}}$ have completed, meaning its start time satisfies $s_{a_g} \geq \max_{k \in \mathcal{S}} f_k$.

Its finish time is given by $f_{a_g} = s_{a_g} + \tau^{\text{agg}}(A, \mathcal{S})$. Downstream consumers of the aggregation, such as the first-layer tasks of the next stage, are then constrained to begin only after the aggregation node completes:

$$s_j \geq f_{a_g} + \delta_{a_g \rightarrow j} \quad (6)$$

e) Execution Semantics and Task Dependencies: Each node $v \in \mathcal{V}$ in the execution graph represents either a task t_i or a collective aggregation node a_g . Let s_v and f_v denote the start and finish times of node v , respectively.

For a computation task t_i , the finish time is given by:

$$f_i = s_i + \tau_i^{\text{comp}} + \tau_i^{\text{comm}} \quad (7)$$

For an aggregation node a_g , which coordinates a collective over a chiplet subset $\mathcal{S}$ using algorithm A , the execution proceeds only after all contributing tasks have completed:

$$s_{a_g} \geq \max_{k \in \mathcal{S}} f_k, \quad f_{a_g} = s_{a_g} + \tau^{\text{agg}}(A, \mathcal{S}) \quad (8)$$

The execution semantics are governed by a dependency DAG $G = (\mathcal{V}, \mathcal{E})$, where an edge $(v_i, v_j) \in \mathcal{E}$ implies:

$$s_j \geq f_i + \delta_{i \rightarrow j} \quad (9)$$

The delay term $\delta_{i \rightarrow j}$ accounts for data availability, buffer flush latency, or explicit synchronization.

f) Optimization Objective: Our primary scheduling objective is to minimize the makespan across all execution nodes, including both computation and aggregation:

$$F = \min_S \max_{v \in \mathcal{V}} f_v \quad (10)$$

This formulation unifies compute heterogeneity, fine-grained DMA delays, collective communication costs, and task dependencies within a single scheduling framework, enabling analysis and optimization of MCM AI accelerators under realistic constraints like bandwidth bottlenecks and controller contention.

B. Dynamic Programming-based Baseline Scheduler

In the baseline formulation without DMA coalescing, we assume that each task $v \in \mathcal{V}$ (including both computation and aggregation tasks) is pre-mapped to a specific PE $p_v = \pi(v)$ at compile time. This mapping is typically obtained through gemini mapper [18]. Given a fixed mapping, the scheduling problem reduces to computing the earliest feasible start and finish times of all nodes, considering data dependencies and communication delays.

Let s_v and f_v denote the start and finish times of node v . The DP recurrence is defined as:

$$f_v = s_v + \tau_v, \quad s_v = \max_{u \in \text{pred}(v)} (f_u + \delta_{u \rightarrow v}) \quad (11)$$

The execution latency τ_v depends on the node type:

- If v is a compute task, then $\tau_v = \tau_v^{\text{launch}} + \tau_v^{\text{comp}}$
- If v is an aggregation node, then $\tau_v = \tau_v^{\text{agg}}$

a) DMA Launch Modeling: Without coalescing, each compute task individually initiates its DMA transfer. The DMA launch overhead is:

$$\tau_v^{\text{launch}} = H_v \cdot \lambda_{\text{hop}} + q_v \cdot \bar{\tau}_{\text{launch}} + \gamma \quad (12)$$

where H_v is the PE-to-controller hop count, q_v is the number of queued descriptors, and γ is the controller setup cost.

The communication latency between dependent tasks $u \rightarrow v$ is implicitly captured by the dependency delay $\delta_{u \rightarrow v}$, which includes data readiness and synchronization delays.

b) Complexity: The scheduler processes the DAG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ in topological order. Since each state depends only on its immediate predecessors, the total complexity is $\mathcal{O}(|\mathcal{E}|)$, offering an efficient baseline with fixed placement and uncoalesced transfers.

C. Extension: PE Address Mapping and Coalesced DMA Communication

In chiplet-based architectures, the physical layout of PEs determines memory routing paths and local aggregation capabilities. We extend the baseline model to incorporate an explicit *address mapping* strategy that enables *coalesced DMA launches*, significantly reducing controller queue pressure and improving the bandwidth efficiency of DNN and LLM execution.

a) PE Grouping via Address Mapping: Let each computing core contain P physical PEs, labeled $\{1, 2, \dots, P\}$. An address mapping partitions these into k disjoint, physically contiguous blocks:

$$\begin{aligned} \mathcal{M} &= \{B_1, B_2, \dots, B_k\} \\ \text{where } B_i &\subseteq \{1, 2, \dots, P\} \\ B_i \cap B_j &= \emptyset \quad \text{for } i \neq j \end{aligned} \quad (13)$$

Each block B_j represents a set of PEs with a shared memory address space and localized routing. For instance, $\mathcal{M} = \{[1-4], [5-8], [9-12], [13-16]\}$ defines $k = 4$ blocks.

b) Aggregation Feasibility under Address Mapping: Address blocks constrain collective operations. Let $\text{group}(p)$ denote the address block containing PE p . Aggregation for task t_i is only permitted if all source PEs belong to the same block:

$$\tau_i^{\text{agg}} = \begin{cases} \tau^{\text{agg}}(A, \mathcal{S}_i), & \text{if } \forall p_j \in \mathcal{S}_i, \text{ group}(p_j) = \text{group}(p_i^{\text{src}}) \\ \infty, & \text{otherwise} \end{cases} \quad (14)$$

This models physical limitations in collective hardware structures.

c) Coalesced DMA Launch Enabled by Address Mapping: Without coalescing, each task independently initiates a DMA transfer with a separate descriptor. Under address mapping, the DMA controller associated with block B_j can allocate a single descriptor for the tasks mapped to B_j , thereby effectively merging multiple DMA requests from $\mathcal{G}_j \subseteq \mathcal{V}$ into one compound descriptor.

We update the DMA launch latency model accordingly:

$$\tau_i^{\text{launch}} = \gamma + I_i \eta g_j + \bar{\tau}_{\text{queue}} - I_i (1 - h_{i,j}) \bar{\tau}_{\text{generate}}, \quad (15)$$

Here:

- τ_i^{launch} denotes the effective DMA launch latency experienced by task i .
- γ is the fixed controller-side descriptor handling latency (per DMA launch), independent of the data size.
- $I_i \in \{0, 1\}$ is a coalescing indicator, where $I_i = 1$ if task i belongs to a coalesced group B_j , and $I_i = 0$ otherwise.
- η is the incremental cost per coalesced task (with $\eta \ll \gamma$), and is set to 10 ns in our simulations to reflect the latency of a simple on-chip communication.
- $g_j = |\mathcal{G}_j|$ denotes the number of coalesced tasks in group B_j .
- $\bar{\tau}_{\text{queue}} = q_j \cdot \bar{\tau}_{\text{launch}}$ models the average queuing delay for group B_j , where q_j is the average number of outstanding

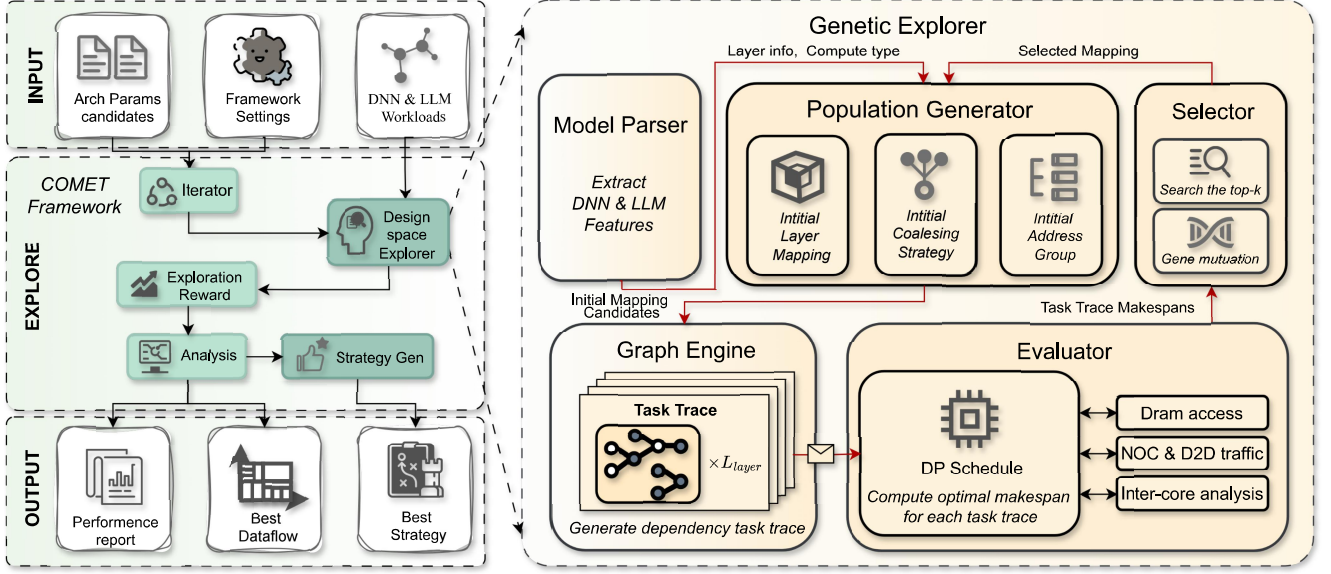


Fig. 4. Overview of COMET Framework. The red arrow illustrates the Exploration iteration loop

DMA launches observed ahead of B_j , and $\bar{\tau}_{\text{launch}}$ is the average launch time per DMA.

- $h_{i,j} \in \{0, 1\}$ is a head-task indicator, where $h_{i,j} = 1$ if task i is the head (first finishing) task in coalesced group B_j , and $h_{i,j} = 0$ otherwise.
- $\bar{\tau}_{\text{generate}}$ denotes the average memory-allocation and interrupt-handling latency.

Within each coalesced group, the head task incurs the full Launch latency γ , whereas the effective launch latency of the remaining tasks is reduced by $\bar{\tau}_{\text{generate}}$ because the buffer space for the entire group has already been allocated.

This amortizes DMA control overhead across multiple transfers and reduces queue occupancy.

d) Routing-Aware Communication Latency: Address grouping also affects communication transmission delay. We extend the transmit latency from Eq. (3) to:

$$\tau_i^{\text{transmit}} = \alpha + \lambda(p_i^{\text{src}}, p_i^{\text{dst}}) \cdot \beta N_i \quad (16)$$

where the scaling factor $\lambda(p, q)$ models interconnect locality:

$$\lambda(p, q) = \begin{cases} 1, & \text{if } \text{group}(p) = \text{group}(q) \\ \lambda_{\text{inter}}, & \text{otherwise} \end{cases} \quad (17)$$

with $\lambda_{\text{inter}} > 1$ representing increased latency or congestion in cross-block routing.

e) Scheduling Optimization under Address Mapping:

Given a fixed mapping $\pi(t)$ and grouping $\mathcal{M}$, scheduling aims to minimize makespan (Eq. (10)) under:

- Aggregation feasibility constraints (Eq. (14)),
- Modified DMA launch latency (Eq. (15)),
- Routing-aware transmission latency (Eq. (16)),
- Resource and dependency constraints (Sec. V-A).

This setup creates a co-design loop between physical placement (hardware address layout) and software-level scheduling.

f) Computational Complexity: The scheduling problem has two stages: (1) Mapping involves assigning PEs a worst-case space of $\mathcal{O}(k^n)$ for k address blocks and n tasks. (2) Scheduling, given a mapping, is NP-hard due to dependency, communication constraints, and non-linear latency from DMA and aggregation. Exact solutions are intractable for large n , requiring heuristics or approximations.

D. COMET Workflow

The COMET framework, illustrated in Figure 4, optimizes task mappings and scheduling for DNNs and LLMs to minimize the overall makespan (i.e., the total execution time of all tasks). Its workflow comprises three phases, which are detailed in the following sections.

1) INPUT Phase: In this phase, COMET collects all essential parameters to initialize the optimization process, as shown in Figure 4. These inputs are then leveraged to drive the exploration of the design space in the next phase.

2) EXPLORE Phase: The EXPLORE phase employs a hardware-aware genetic algorithm to efficiently explore the large mapping and scheduling space. The process includes three main steps: initial population construction, exploration strategies, and evolutionary search.

a) Initial Population Construction: We begin by generating an initial population of P candidate configurations. (1) For each individual, the mapper constructs an initial layer mapping $\mathcal{L}$ based on the workload and system configuration, which is then converted into the task-to-PE mapping π_i and task DAG $\mathcal{T}_i$ using the communication graph G . Address clustering $\mathcal{M}_i$ and DMA coalescing configuration $\mathcal{C}_i$ are also initialized. (2) Tasks t_j in the DAG $\mathcal{T}_i$ are scheduled in topological order. For each candidate PE p , we estimate the total execution latency, including computation, communication, and launch delays. The communication delay considers transfer latency

between tasks, while the launch delay models contention and coalescing efficiency. (3) The best candidate PE p^* minimizing the finish time f_j is selected, and the complete timing schedule S_i is recorded. The fitness of each individual is then evaluated as the makespan.

b) Exploration Strategies: COMET optimizes end-to-end latency through a multi-stage process: (1) it partitions the model across chiplets based on system parameters, model weights, and memory access patterns to achieve a balanced workload distribution; (2) within each chiplet, the model is further divided and assigned to cores to determine the task-to-PE mapping; (3) within each core, different address grouping strategies are explored to optimize data locality and communication efficiency.

- **COMET-Row:** This strategy divides the core into rows, with the smallest unit being half a row and the largest unit covering all rows. PEs are grouped along the row axis to enhance parallelism and ensure efficient data transfer.
- **COMET-Column:** This strategy divides the core into columns, with the smallest unit being half a column and the largest unit covering all columns. It optimizes workloads that benefit from column-based data access.
- **COMET-Other:** This strategy allows for flexible combinations, grouping PEs in both row- and column-based units.

These strategies are applied to each chiplet's PEs to identify efficient task mappings. Although each layer may use different combinations, the partitioning remains consistent within each chiplet. To avoid inefficiency from excessive flexibility, COMET adopts structured partitioning schemes in the final design.

c) Evolutionary Search: The genetic search runs for $G_{\max}$ generations with two main steps:

(1) **Beam-Based Refinement:** The evaluator selects the top- K individuals with the shortest completion times. Local search is conducted in three forms, each refining both task-to-PE mapping and PE address grouping: (1) a risk-taking search that explores highly divergent mapping and grouping patterns within the current exploration rules to enhance diversity and avoid local optima; (2) an observation-driven exploratory search that adjusts mappings and groupings based on performance feedback and PE idle-time analysis across chiplets, where PEs with higher idle ratios use smaller address groupings or are assigned heavier workloads to improve utilization; and (3) a reference-based search that leverages optimized configurations from similar models on the same system to guide parameter refinement. Each refined individual is rescheduled and re-evaluated, and both refined and original top- K individuals form the parent pool for the next generation.

(2) **Structured Crossover and Mutation:** New individuals are generated to replenish the population. A fraction determined by the mutation rate μ undergoes mutation before crossover to increase population diversity. For each new child, two parents are randomly selected from the parent pool, and their configurations $(\pi, \mathcal{M}, \mathcal{C})$ are combined through structured crossover, where the task-to-PE mapping π and address clustering $\mathcal{M}$

are treated as a bound pair and merged structurally to preserve useful substructures and maintain locality. For each parent pair (P_1, P_2) , four children are produced, including two identical to the parents and two crossover offspring generated by exchanging the coalescing configurations while keeping $(\pi, \mathcal{M})$ unchanged:

$$\begin{aligned} P_1 &= (\pi_1, \mathcal{M}_1, \mathcal{C}_1), & P_2 &= (\pi_2, \mathcal{M}_2, \mathcal{C}_2), \\ O_1 &= (\pi_1, \mathcal{M}_1, \mathcal{C}_1), & O_2 &= (\pi_2, \mathcal{M}_2, \mathcal{C}_2), \\ O_3 &= (\pi_1, \mathcal{M}_1, \mathcal{C}_2), & O_4 &= (\pi_2, \mathcal{M}_2, \mathcal{C}_1). \end{aligned}$$

Each resulting child is subsequently rescheduled and re-evaluated to determine its fitness.

3) OUTPUT Phase: After all generations are completed, COMET outputs the best-performing configuration $(\pi^*, \mathcal{M}^*, \mathcal{C}^*)$ along with the corresponding schedule S^* . The OUTPUT phase provides the optimal task mapping, the selected clustering and DMA coalescing configuration, and performance metrics, including the final makespan and resource utilization.

VI. EVALUATION

A. Simulation

To evaluate the performance gains of COMET, we utilize a simulation framework [48] that integrates SCALE-Sim [47] for modeling deep learning inference workloads and Booksim [43] for simulating the interconnection network. We extend this framework to implement various versions of the COMET algorithms proposed in this work. We further enhance the simulation framework to support DMA-based inter-chiplet communication [36], [37], enabling more realistic modeling of data movement. In our simulations, the fixed launch time γ and the DMA descriptor-handling latency $\bar{\tau}_{\text{launch}}$ are set to 40 μs and 2 μs , following [38]. The overall system configuration used in our experiments is summarized in Table I. Each Chiplet in the system consists of 4×4 Computing Cores, and each Computing Core in the system consists of a 4×4 array of PEs, with each PE incorporating a 64×64 MAC array. The perturbed factor of each PE, set to 0.3, follows the distribution adopted in previous work [29]. The computational power of each Chiplet matches that of prior work [48], [51], ensuring consistency with earlier performance evaluations. We assumed FP16 inferencing for all experiments. The interconnect is modeled as a mesh topology with homogeneous bidirectional links between neighboring chiplets with UCIE phy, each providing 32 GB/s of bandwidth. The system operates at a clock frequency of 1 GHz.

B. AI Workload

To demonstrate the effectiveness and versatility of COMET, we evaluate the system performance across various real-world deep neural network models from SCALE-Sim, covering the inference scenarios. The models include AlexNet [52], AlphaGoZero [53], FasterRCNN [54], GoogLeNet [55], NCF-Recommendation (NCF) [56], ResNet152 [57], and Transformer [5]. We also include GPT-2 [58], Llama3 [3] and ViT

TABLE I
SYSTEM CONFIGURATION

Parameters	Configurations
Chiplet	Number of Computing Cores
	4×4
Core	Clock Frequency
	1GHz
Core	Number of PEs
	4×4
Core	Clock Frequency
	1GHz
PE	MAC Array Height
	64, 32, 16
	MAC Array Weight
	64, 32, 16
	ifmap SRAM
	2048, 1024, 512
PE	filter SRAM
	2048, 1024, 512
	ofmap SRAM
	2048, 1024, 512
	Precision
	16 bits
Network	Dataflow
	Output Stationary
	Bandwidth
	32GBps
	Packet Size
	8192B
	Flit Size
	512B
	Per Flit Latency
	21ns
Network	Router Clock Frequency
	1GHz
	Topology
	Mesh
	Flow Control
	Virtual Cut-Through
Network	Number of VCs
	4
Network	VC Buffer Depth
	318 flits

[59] workloads to verify the adaptability and performance of our schemes on large-scale language models. All models are under FP16 data type. To better align with inference settings, the mini-batch size is fixed at 4 throughout all experiments. The COMET framework does not alter neural network computation ordering and thus does not affect model accuracy or convergence.

C. Benchmarks

We evaluate two categories of models, referred to as Flexible Mode and Strict Mode. In *Flexible Mode*, only partial information from the previous layer is needed (e.g., Transformer-based models), while *Strict Mode* requires complete output from the previous layer (e.g., CNN-based models). To compare the performance under both modes, we modify the Transformer computation graph to follow Strict Mode.

We evaluate several collective communication algorithms, including Ring, Ring-2D, DTree, RingBi, and TTO [48]. For DMA operations, we use full coalescing, as the computation variance across nodes is minimal in aggregation scenarios, ensuring a fair comparison. The Baseline approach follows Sec. V-B, where each PE independently issues DMA requests without aggregation. The Coalescing method uses full coalescing, while the remaining approaches are derived through the COMET framework: COMET-Row, COMET-Column, and COMET-Other. The latter combines partial row and column coalescing (e.g., 1/2 or 1/4), based on empirical observations. The total runtime is divided into four parts: Compute, Comm, Compute dma, and Comm dma. Compute is the time spent on computation, Comm on communication, Compute dma on DMA triggered by computation, and Comm dma on DMA during aggregation and communication. After computation, all DMA requests from PEs within a chiplet are aggregated for communication.

D. End-to-end Performance

All experiments in this section share a unified configuration unless otherwise noted:

- Mapping strategies: Baseline, Coalesced, COMET-R, COMET-C, and COMET-O.
- Communication algorithms: Ring, Ring-2D, DTree, RingBi, and TTO.
- Runtime decomposition: Each bar shows the runtime breakdown into Compute, Comm, Compute dma, and Comm dma.
- Speedup: End-to-end inference speedup is normalized to the baseline and is indicated by black lines or dots.

1) *Runtime Trends in Strict Mode Models*: We first compare COMET’s end-to-end performance in Strict Mode Models against the baseline. As shown in Figure 5, the Coalesced and COMET strategies achieve a speedup of $1.1\times$ to $2.6\times$ over the baseline. A key observation is the significant reduction in DMA-related runtime (comm dma and compute dma) with optimized mapping strategies. Compared to the baseline, these strategies notably lower DMA overhead, especially under collective algorithms like Ring-2D and TTO. For models such as ResNet152, dlrm, and Transformer, the reduction in comm dma exceeds 90% under TTO, leading to up to $2.4\times$, $2.2\times$, and $2.5\times$ speedup, respectively. For compute-bound workloads like AlexNet, NCF, GoogLeNet, and FasterRCNN, the speedup ranges from $1.5\times$ to $2\times$.

2) *Runtime Trends in Flexible Mode Models*: Figure 6 shows the normalized inference runtime and end-to-end speedup for four Transformer-based models (Llama3 7B, Transformer, GPT-2, and ViT) across five communication algorithms and mapping strategies. Llama3 uses the GQA method, while the others use MHA. For Llama3 and Transformer, runtime remains consistent across mappings, indicating they are compute-bound, with minimal communication overhead. Speedups are modest (mostly $1.0\times$ to $1.2\times$), with the Compute component dominating. GPT-2 and ViT are more sensitive to DMA and communication overheads, particularly in the Compute dma segment, where optimized mapping strategies like COMET-O reduce DMA time and improve speedup, reaching up to $1.6\times$ compared to the baseline. The impact is particularly notable for ViT. Among the COMET variants, COMET-C and COMET-O outperform COMET-R, especially on ViT. COMET-C achieves up to $1.4\times$ speedup, and COMET-O up to $1.6\times$ under Ring-2D and TTO algorithms. Their column-based mapping strategy reduces inter-PE communication and improves pipeline efficiency, while COMET-R’s row-based aggregation increases dependencies and synchronization delays, leading to lower PE utilization.

E. Impact of Execution Mode on Transformer Inference

In this section, we compare the *Strict Mode* (Figure 5) and the *Flexible Mode* (Figure 6) for the Transformer model to reveal a striking shift in performance characteristics and optimization sensitivity. In the *Strict Mode*, where layer-by-layer synchronization introduces serialized communication and stronger inter-layer coupling, especially with model partitions across separate PEs or chiplets, data movement, particularly compute DMA, significantly impacts performance. Optimized mappings like COMET-C and COMET-O show

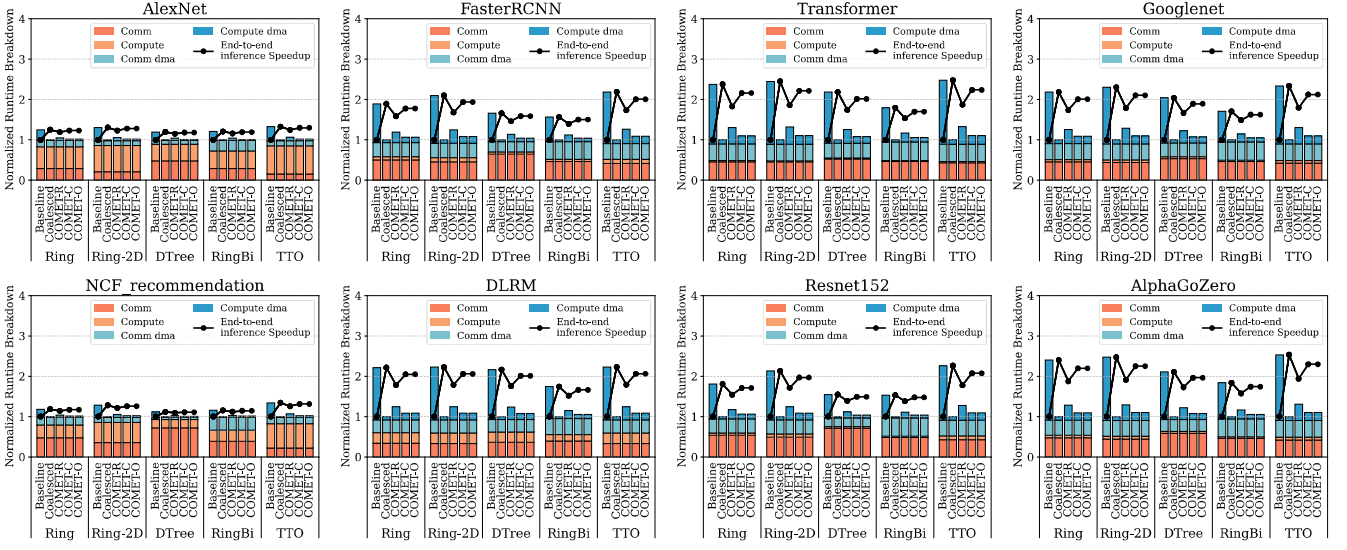


Fig. 5. Inference Time Breakdown in Strict Mode for 8×8 Mesh

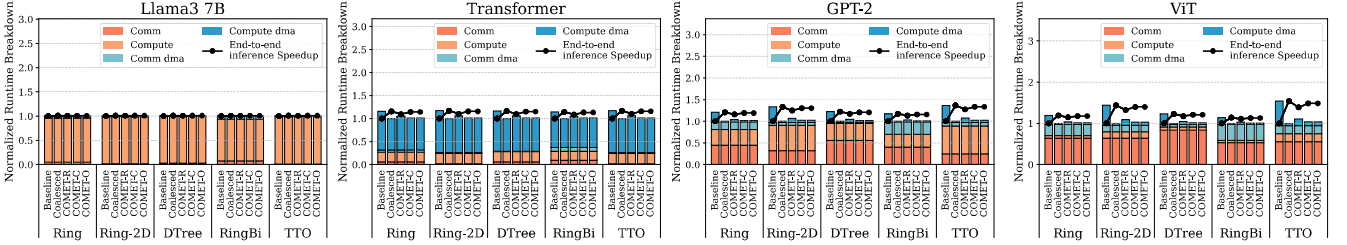


Fig. 6. Inference Time Breakdown in Flexible Mode for 8×8 Mesh

more substantial gains, with speedups reaching nearly $2.8\times$ compared to the baseline, driven by significant reductions in DMA overhead. In the *Flexible Mode*, where attention layers run independently, the runtime is primarily compute-bound, with minimal communication and DMA overhead across topologies and mapping strategies. As a result, COMET achieves limited speedup, typically within $1.2\times$.

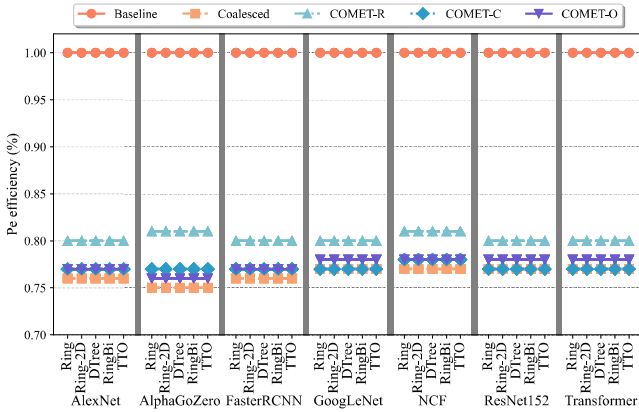


Fig. 7. PE Efficiency for 8×8 Mesh

F. Impact of memory mapping

To analyze PE efficiency variations in Figure 7, we observe multiple task-to-processor mapping strategies across diverse

models. The Baseline, with a PE efficiency of 1, allows a PE to initiate a DMA request immediately after completing its computation. The Coalesced strategy consistently shows lower PE efficiency, ranging from 0.75 to 0.78.

The COMET series outperforms Coalesced in PE efficiency. For instance, COMET-R achieves around 0.80 efficiency across models, indicating that memory-aware mapping improves PE utilization by better pipelining and overlapping computation with data movement. This highlights a key trade-off: while the Coalesced approach can reduce end-to-end runtime, it sacrifices PE utilization due to strict layer-wise synchronization. The Coalesced method causes PEs to idle while waiting for the slowest PE to complete, significantly underutilizing resources.

The PE efficiency shown here is computed as:

$$\text{PE Efficiency} = \frac{T_{\text{comp}}}{T_{\text{dma_ready}} - T_{\text{comp_start}}} \quad (18)$$

This metric reflects the fraction of active execution time within the total DMA-to-compute window. Therefore, higher values indicate better compute overlap and reduced idle periods.

G. Bandwidth Utilization

As shown in Figure 8, The COMET series (COMET-R, COMET-C, COMET-O) consistently achieves higher bandwidth utilization than the *Baseline*, with only marginal im-

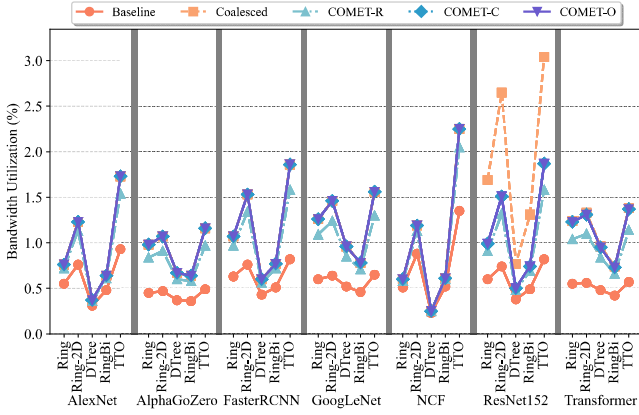


Fig. 8. Bandwidth Utilization for 8×8 Mesh

provement over *Coalesced*, and slightly lower performance on the ResNet152 workload. COMET-O demonstrates the most robust performance, with utilization peaking above 2% on NCF (TTO) and ResNet152 (TTO), and reaching up to $\sim 3\%$ for ResNet152 under Ring2D, indicating strong adaptability to data-intensive workloads. In contrast, the *Baseline* remains largely below 1% often falling under 0.5% for models such as AlphaGoZero and Transformer.

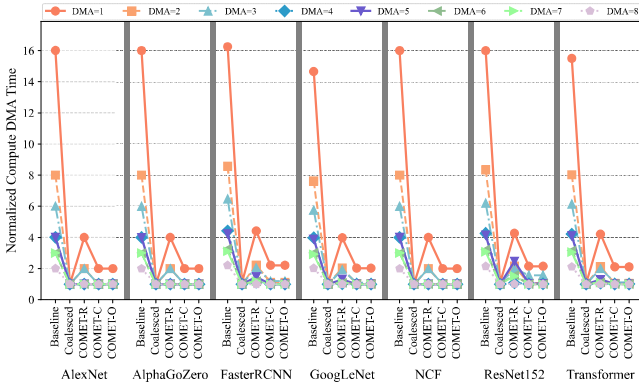


Fig. 9. Normalized Compute DMA Time for Varing DMA Controller Count

H. Scalability result

Figure 9 presents the scalability of various deep learning workloads with respect to the number of DMA controllers ranging from 1 to 8, with each PE configured with 64×64 MAC arrays on the 8×8 mesh. As the number of DMA controllers increases, all configurations exhibit performance improvements due to higher parallelism in data transfer. The baseline scheme achieves a modest speedup of approximately $1.8\times$ to $2.5\times$ across the tested models. In contrast, the optimized memory access schemes scale more effectively, yielding speedups of up to $8\times$, attributed to reduced memory contention and better utilization of available bandwidth. Despite scaling hardware parallelism by a factor of 8, the baseline configuration consistently underperforms: its normalized computation time remains 40%–60% higher than that of the best-

optimized schemes at the maximum DMA count. This persistent performance gap indicates that architectural inefficiencies, particularly in memory access granularity and coherence, limit scalability regardless of hardware augmentation.

I. Hardware Evaluation of DMA

To characterize DMA behavior on real hardware, we conduct experiments on an Intel Agilex® 7 I-Series device. Due to the encrypted and proprietary nature of the DMA IP, detailed internal timing is not accessible, so we infer latency components by measuring the end-to-end latency of a single DMA descriptor for varying data sizes, as shown in Figure 10. The results indicate that the DMA processing overhead (blue bars) remains constant across small data sizes, while the transfer time (red bars) increases with data size, suggesting that processing overhead is independent of data size and transfer time grows modestly with size.

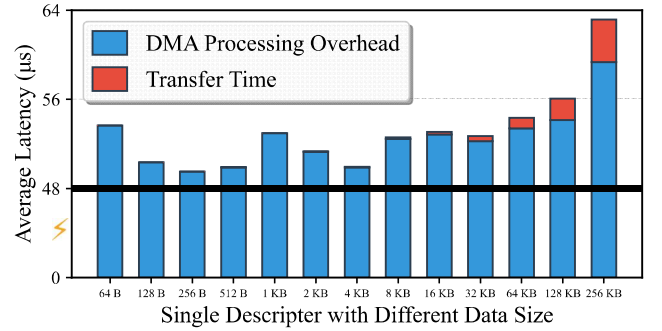


Fig. 10. Average DMA Latency Comparison on Intel Agilex® 7 I-Series

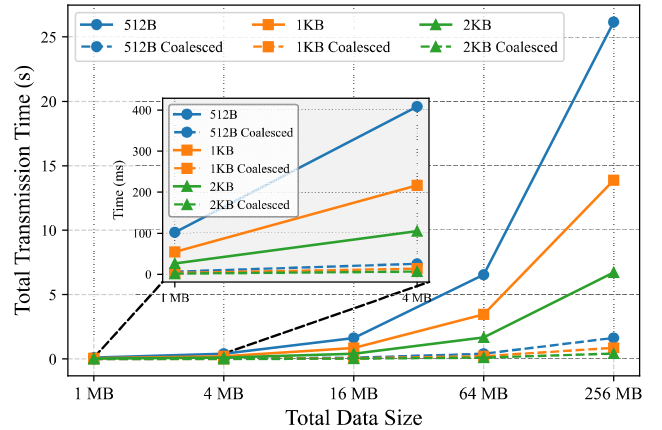


Fig. 11. Transmission Time for Different PE Output Buffer Sizes with Varying Data Sizes on Intel Agilex® 7 I-Series

To analyze DMA launch latency scaling with coalescing, we examine the data in Figure 11 using three PE output buffer sizes (512 B, 1024 B, and 2048 B), keeping the total transferred data constant. DMA coalescing groups up to 16 requests per core into a single launch, excluding PE computation time. With a fixed total data size, any variation in transmission time reflects the DMA launch overhead. The results show that coalesced transmission significantly reduces total time

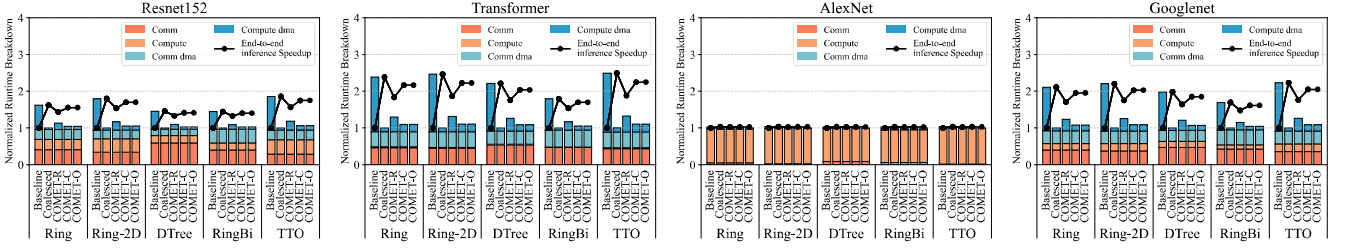


Fig. 12. Inference Time Breakdown in Strict Mode with 16×16 MAC Array

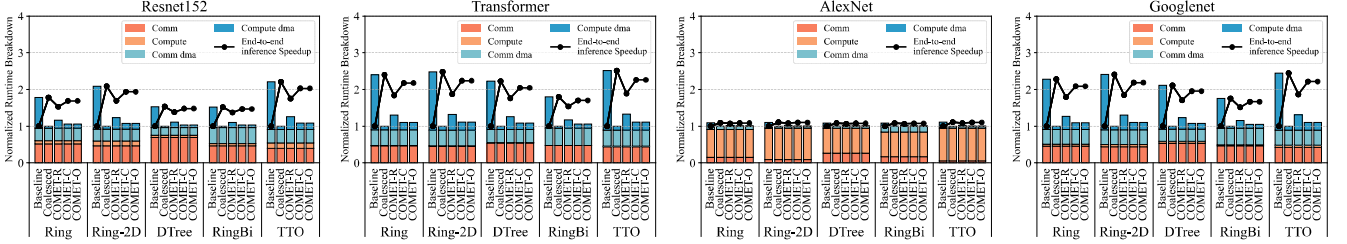


Fig. 13. Inference Time Breakdown in Strict Mode with 32×32 MAC Array

compared to packet-by-packet transmission, confirming that DMA launch latency is effectively amortized on the Intel Agilex® 7 I-Series.

J. Impact of different Architecture

In this section, we evaluate COMET using the MCM-based Simba [14] accelerator, featuring 36 Computing Cores in a 6×6 mesh, each with 16 PEs and an 8×8 MAC array. However, we focus on 16×16 and 32×32 MAC arrays, as smaller 8×8 MAC arrays are less suitable for data-parallel inference due to their limited compute power and higher overhead. For the 16×16 and 32×32 arrays, as shown in Figure 13 and Figure 12, COMET achieves speedups of $1.11 \times$, $2.51 \times$, $2.44 \times$, and $2.2 \times$ over the baseline for AlexNet, Transformer, GoogLeNet, and ResNet152, respectively. In contrast, smaller MAC arrays yield lower speedups, with compute-intensive models like AlexNet experiencing a slight reduction in speedup due to the higher computational load that smaller arrays struggle to handle efficiently.

VII. CONCLUSION

In this paper, we propose COMET, a communication and memory co-design framework to address key performance bottlenecks in MCM accelerators. COMET tackles challenges like fine-grained bursty DMA traffic and static address mappings by introducing techniques to optimize inter-chiplet data movement. We develop a detailed DMA communication model that accounts for protocol overhead, controller contention, and hop delays, and design a coalesced DMA mechanism that dynamically aggregates small transfers to reduce overhead and improve throughput. We also propose an adaptive on-chip memory mapping strategy that aligns data placement with workload behavior, reduces stalls, and enhances data reuse. By extending existing simulators with DMA modeling, we uncover load imbalances and congestion patterns

missed by traditional tools. Evaluation shows that COMET significantly improves makespan, bandwidth efficiency, and PE efficiency across AI workloads, laying the foundation for future hardware/software co-optimization in chiplet-based AI accelerators.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all the reviewers for their valuable feedback. Our deepest appreciation goes to our shepherd for the insightful and constructive guidance, which has been instrumental in helping us refine and enhance the quality of our work, making it more comprehensive and impactful. We would like to thank Tianhe high-performance computing team leaders and members. We thank Zhao Wang, Xiping Dong, and Jinwei Zhou for their valuable discussions and simulation assistance. A special thanks to Professor Lizhou Wu for his support for the hardware Evaluation. This work is supported by the National Key Research and Development Program of China (Grant No.2022YFB4501702) and the National Natural Science Foundation of China (Grant No.62421002 and No.U24B20151). This work is supported by Beijing Natural Science Foundation (Grant No. L243001) and 111 Project (B18001). Dezun Dong and Guangyu Sun are the corresponding authors of this paper.

REFERENCES

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Proceedings of the 34th Conference on Neural Information Processing Systems (NIPS)*, 2020, pp. 1877–1901.
- [2] OpenAI, “Gpt-4 technical report,” 2023.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” in *arXiv preprint arXiv:2302.13971*, 2023.

- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 21st Conference of the North American chapter of the association for computational linguistics (NAACL)*, 2019, pp. 4171–4186.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS)*, 2017.
- [6] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proceedings of the 30th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1492–1500.
- [7] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI)*, 2017, pp. 4278–4284.
- [8] D. Dong, Z. Wang, and F. Lei, "Zettafly: A network topology with flexible non-blocking regions for large-scale ai and hpc systems," in *Proceedings of the 52nd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 835–848.
- [9] Y. Jiao, L. Han, R. Jin, Y.-J. Su, C. Ho, L. Yin, Y. Li, L. Chen, Z. Chen, L. Liu *et al.*, "7.2 a 12nm programmable convolution-efficient neural-processing-unit chip achieving 825tops," in *Proceedings of the 67th IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 136–140.
- [10] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma *et al.*, "Ten lessons from three generations shaped google's tpuv4i: Industrial product," in *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.
- [11] G. E. Moore *et al.*, "Cramming more components onto integrated circuits," 1965.
- [12] S. Naffziger, N. Beck, T. Burd, K. Lepak, G. H. Loh, M. Subramony, and S. White, "Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families: Industrial product," in *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 57–70.
- [13] P. Huang, C. Lu, W. Wei, C. Chiu, K. Ting, C. Hu, C. Tsai, S. Hou, W. Chiou, C. Wang *et al.*, "Wafer level system integration of the fifth generation cowos@s with high performance si interposer at 2500 mm²," in *Proceedings of the 71st IEEE Electronic Components and Technology Conference (ECTC)*, 2021, pp. 101–104.
- [14] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the 52nd IEEE/ACM Annual International Symposium on Microarchitecture (MICRO)*, 2019, pp. 14–27.
- [15] E. Talpes, D. D. Sarma, D. Williams, S. Arora, T. Kunjan, B. Floering, A. Jalote, C. Hsiang, C. Poorna, V. Samant *et al.*, "The microarchitecture of dojo, tesla's exa-scale computer," in *IEEE Micro*, vol. 43, no. 3, 2023, pp. 31–39.
- [16] S. Lie, "Cerebras architecture deep dive: First look inside the hardware/software co-design for deep learning," in *IEEE Micro*, vol. 43, no. 3, 2023, pp. 18–30.
- [17] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized coarse-grained dataflow for scalable nn accelerators," in *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 807–820.
- [18] J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma, "Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators," in *Proceedings of the 30th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 156–171.
- [19] intel, "Intel acceleration stack for intel® xeon® cpu with fpgas core cache interface (cci-p) reference manual," 2019.
- [20] C. Hwang, K. Park, R. Shu, X. Qu, P. Cheng, and Y. Xiong, "Ark:gpu-driven code execution for distributed deep learning," in *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023, pp. 87–101.
- [21] A. Evans, C. Fuguet, and D. Million, "Opensource heterogeneous chiplet-based computing architectures," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, pp. 1–8.
- [22] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object detection in 20 years: A survey," in *Proceedings of the IEEE*, 2023, pp. 257–276.
- [23] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, "Object detection with deep learning: A review," in *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, vol. 30, no. 11, 2019, pp. 3212–3232.
- [24] E. Adamopoulou and L. Moussiades, "An overview of chatbot technology," in *Proceedings of the 16th IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI)*, 2020, pp. 373–383.
- [25] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," in *IEEE Journal on Emerging and Selected Topics in Circuits and Systems (ESCS)*, vol. 9, no. 2, 2019.
- [26] C. Zhang, L. Colagrande, R. Andri, T. Benz, G. Islamoglu, A. Nadalini, F. Conti, Y. Li, and L. Benini, "Flatattention: Dataflow and fabric collectives co-optimization for efficient multi-head attention on tile-based many-pe accelerators," in *arXiv preprint arXiv:2505.18824*, 2025.
- [27] N. K. Jha, S. Ravishankar, S. Mittal, A. Kaushik, D. Mandal, and M. Chandra, "Draco: Co-optimizing hardware utilization, and performance of dnn on systolic accelerator," in *Proceedings of the 30th IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2020, pp. 574–579.
- [28] M. Han, L. Wang, L. Xiao, T. Cai, Z. Wang, X. Xu, and C. Zhang, "Redas: A lightweight architecture for supporting fine-grained reshaping and multiple dataflows on systolic array," in *IEEE Transactions on Computers (TC)*, vol. 73, no. 8, 2024, pp. 1997–2011.
- [29] M. Ma, J. Tan, X. Wei, and K. Yan, "Process variation mitigation on convolutional neural network accelerator architecture," in *Proceedings of the 37th IEEE International Conference on Computer Design (ICCD)*, 2019, pp. 47–55.
- [30] D. D. Sharma, G. Pasdast, Z. Qian, and K. Aygun, "Universal chiplet interconnect express (ucie): An open industry standard for innovations with chiplets at package level," in *IEEE Transactions on Components, Packaging and Manufacturing Technology (TCPMT)*, vol. 12, no. 9, 2022, pp. 1423–1431.
- [31] M. James, M. Tom, P. Groeneveld, and V. Kibardin, "Isdp 2020 physical mapping of neural networks on a wafer-scale deep learning accelerator," in *Proceedings of the 29th ACM International Symposium on Physical Design (ISPD)*, 2020, pp. 145–149.
- [32] J. Vasiljevic, L. Bajic, D. Capalija, S. Sokorac, D. Ignjatovic, L. Bajic, M. Trajkovic, I. Hamer, I. Matosevic, A. Cejkov *et al.*, "Compute substrate for software 2.0," in *IEEE Micro*, vol. 41, no. 2, 2021, pp. 50–55.
- [33] A. Yang, "Deep learning training at scale spring crest deep learning accelerator (intel® nervana™ nnp-t)," in *Proceedings of the 31st IEEE Hot Chips Symposium (HCS)*, 2019, pp. 1–20.
- [34] D. Lu and Q. Weng, "A survey of image classification methods and techniques for improving classification performance," in *Proceedings of the 1st International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT)*, 2007, pp. 823–870.
- [35] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proceedings of the 10th IEEE International Workshop on Frontiers in Handwriting Recognition (IWFHR)*, 2006.
- [36] H. Kim, Y. Choi, J. Park, B. Bae, H. Jeong, S. M. Lee, J. Yeon, M. Kim, C. Park, B. Gu *et al.*, "Tep: A tensor contraction processor for ai workloads industrial product," in *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 890–902.
- [37] S. Y. Lee, H. Lee, J. Hong, S. Cho, and J. W. Lee, "Vga: Hardware accelerator for scalable long sequence model inference," in *Proceedings of the 57th IEEE/ACM Annual International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1444–1457.
- [38] T. Jose and D. Shankar, "Performance modeling of a heterogeneous computing system based on the ucie interconnect architecture," in *Proceedings of the 14th IEEE Space Computing Conference (SCC)*, 2023, pp. 5–10.
- [39] J. Su, J. Li, L. Chen, C. Li, K. Zhang, L. Yang, and Y. Xu, "Revitalizing the forgotten on-chip dma to expedite data movement in nvm-based storage systems," in *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST)*, 2023, pp. 363–378.

- [40] N. Vujic, F. Cabarcas, M. G. Tallada, A. Ramirez, X. Martorell, and E. Ayguade, "Dma++: On the fly data realignment for on-chip memories," in *IEEE Transactions on Computers (TC)*, vol. 61, no. 2, 2010, pp. 237–250.
- [41] S. Ma, Z. Liu, S. Chen, L. Huang, Y. Guo, Z. Wang, and M. Zhang, "Coordinated dma: improving the dram access efficiency for matrix multiplication," in *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 30, no. 10, 2019, pp. 2148–2164.
- [42] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, and Saidi, "The gem5 simulator," in *Acm Sigarch Computer Architecture News*, vol. 39, no. 2, 2011, pp. 1–7.
- [43] N. Jiang, J. Balfour, D. U. Becker, B. Towles, W. J. Dally, G. Micheliogiannakis, and J. Kim, "A detailed and flexible cycle-accurate network-on-chip simulator," in *Proceedings of the 14th IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 86–96.
- [44] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "Garnet: A detailed on-chip network model inside a full-system simulator," in *Proceedings of the 10th IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 33–42.
- [45] S. Bharadwaj, J. Yin, B. Beckmann, and T. Krishna, "Kite: A family of heterogeneous interposer topologies enabled via accurate interconnect modeling," in *Proceedings of the 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [46] Y. Feng, Y. Wei, D. Xiang, and K. Ma, "Evaluating chiplet-based large-scale interconnection networks via cycle-accurate packet-parallel simulation," in *Proceedings of the 33rd USENIX Annual Technical Conference (USENIX ATC)*, 2024, pp. 731–747.
- [47] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "Scale-sim: Systolic cnn accelerator simulator," in *arXiv: Distributed, Parallel, and Cluster Computing*, 2018.
- [48] S. Laskar, P. Majhi, S. Kim, F. Mahmud, A. Muzahid, and E. J. Kim, "Enhancing collective communication in mcm accelerators for deep learning training," in *Proceedings of the 30th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 1–16.
- [49] AMD, "Axi dma v7.1," 2025.
- [50] D. Das Sharma, G. Pasdast, S. Tiagaraj, and K. Aygün, "High-performance, power-efficient three-dimensional system-in-package designs with universal chiplet interconnect express," in *Nature Electronics*, vol. 7, no. 3, 2024, pp. 244–254.
- [51] D. Lim and J. Kim, "Tidalmesh: Topology-driven allreduce collective communication for mesh topology," in *Proceedings of the 31st IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 1526–1540.
- [52] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proceedings of the 26th Conference on Neural Information Processing Systems (NIPS)*, 2012.
- [53] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," in *Nature*, vol. 550, no. 7676, 2017, p. 354–359.
- [54] R. Girshick, "Fast r-cnn," in *Proceedings of the 15th IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1440–1448.
- [55] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the 28th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [56] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering *," in *Proceedings of the 26th ACM Conference on World Wide Web (WWW)*, 2017, pp. 173–182.
- [57] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the 29th IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [58] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," in *OpenAI blog*, 2019, p. 9.
- [59] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *arXiv preprint arXiv:2010.11929*, 2020.